

The Perceptron Trick

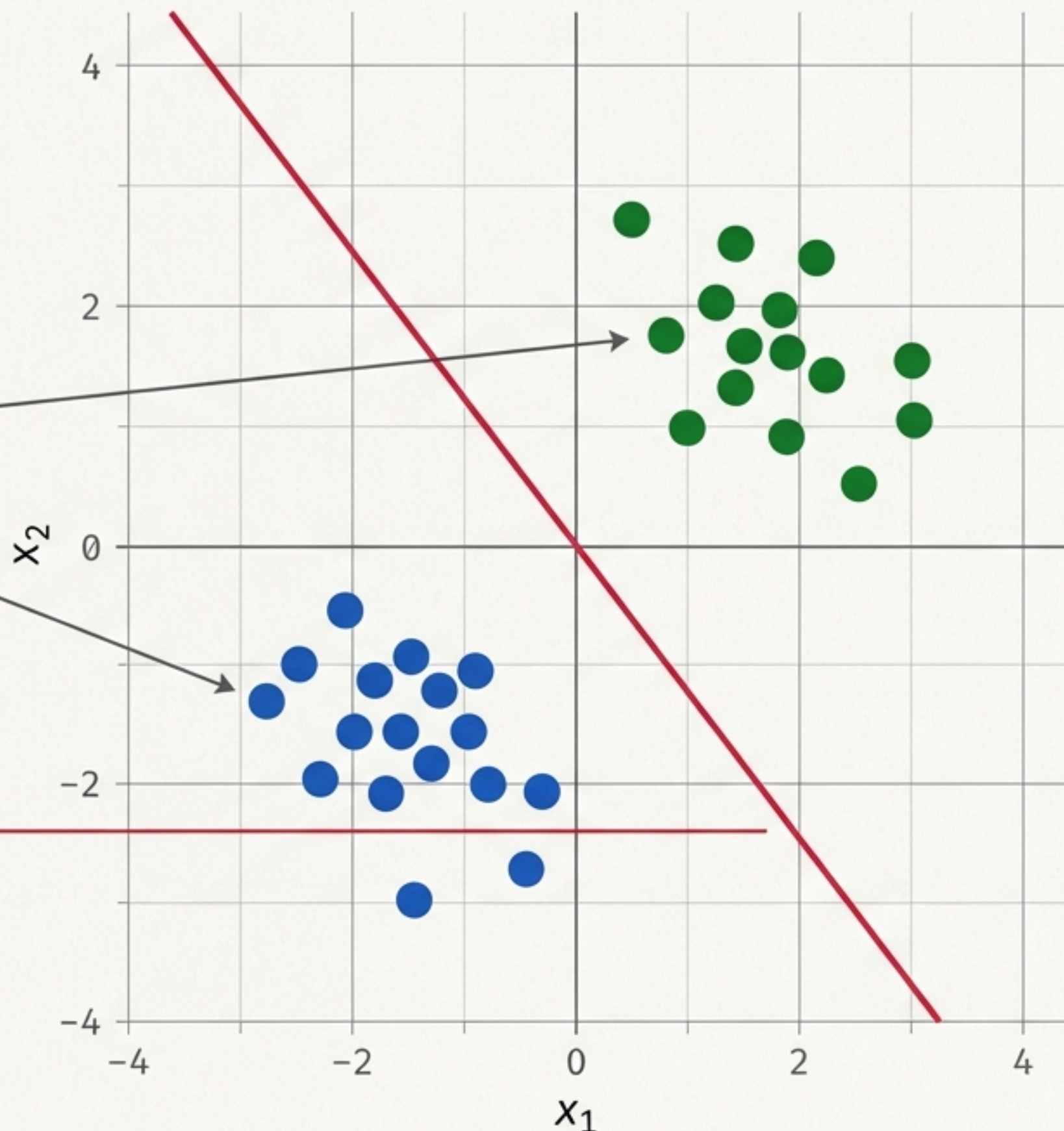
Geometric intuition for Logistic Regression.

Moving from a random guess to a perfect mathematical separator.

This iterative process adjusts the decision boundary (the Crimson Red line) to optimally separate data points based on their labels. Each misclassified point triggers a geometric transformation, rotating and shifting the line until all positive examples are on one side and all negative examples are on the other. The resulting separator perfectly classifies the training data, forming the basis for binary classification in Logistic Regression.

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

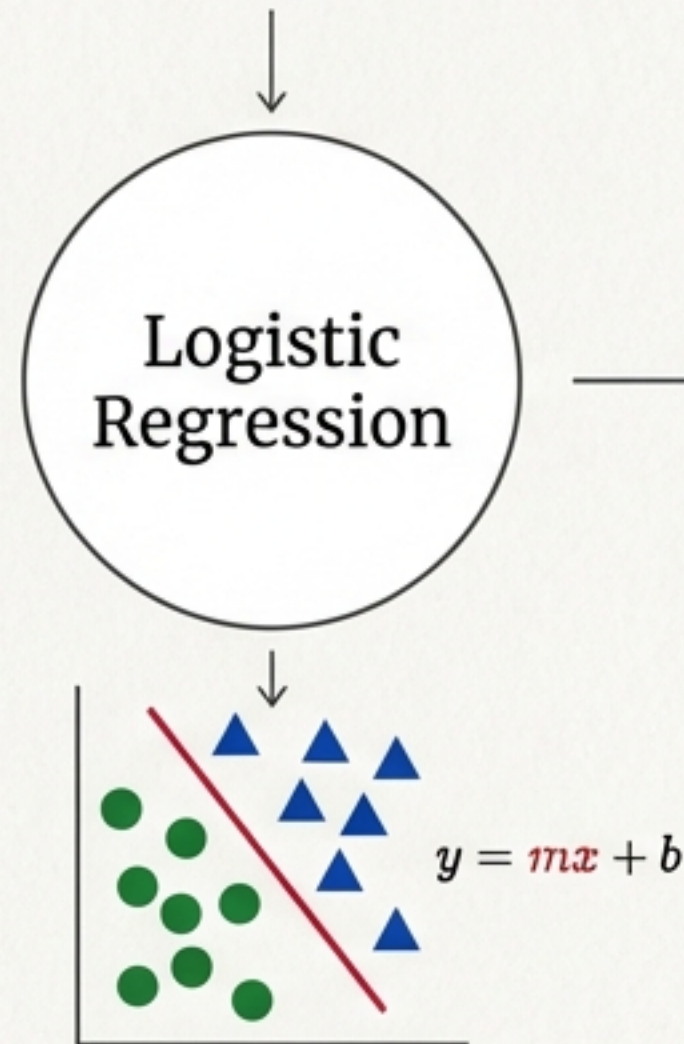
where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the feature vector, and b is the bias term. The line separates regions where $\mathbf{w} \cdot \mathbf{x} + b > 0$ (Kelly Green zone) from $\mathbf{w} \cdot \mathbf{x} + b < 0$ (Sapphire Blue zone).



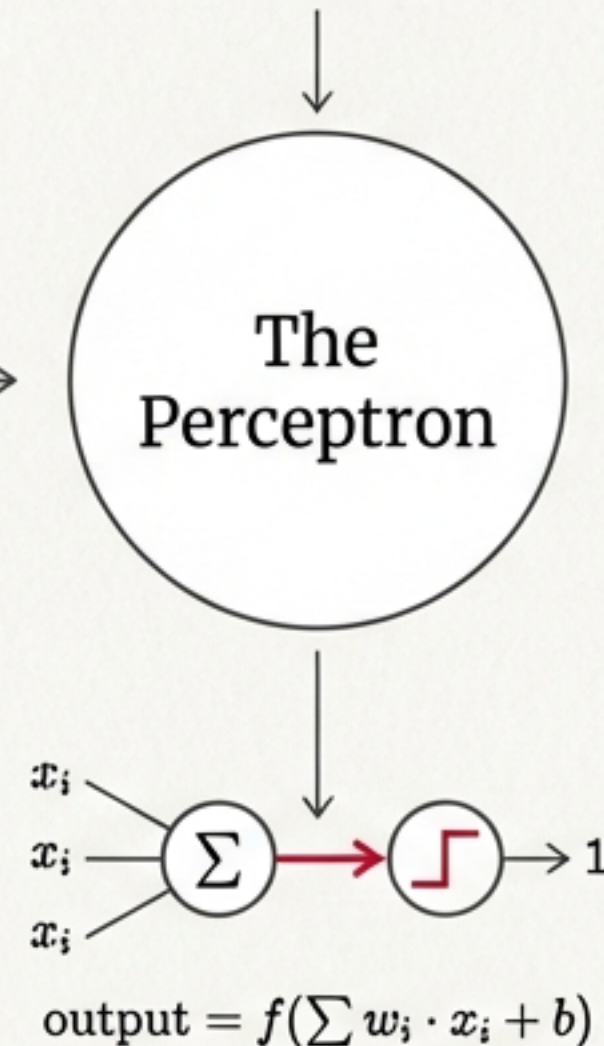
The Foundation of Deep Learning

Mastering this single line in 2D space creates the exact foundation required to understand deep neural networks. The logic scales from two dimensions to infinite dimensions.

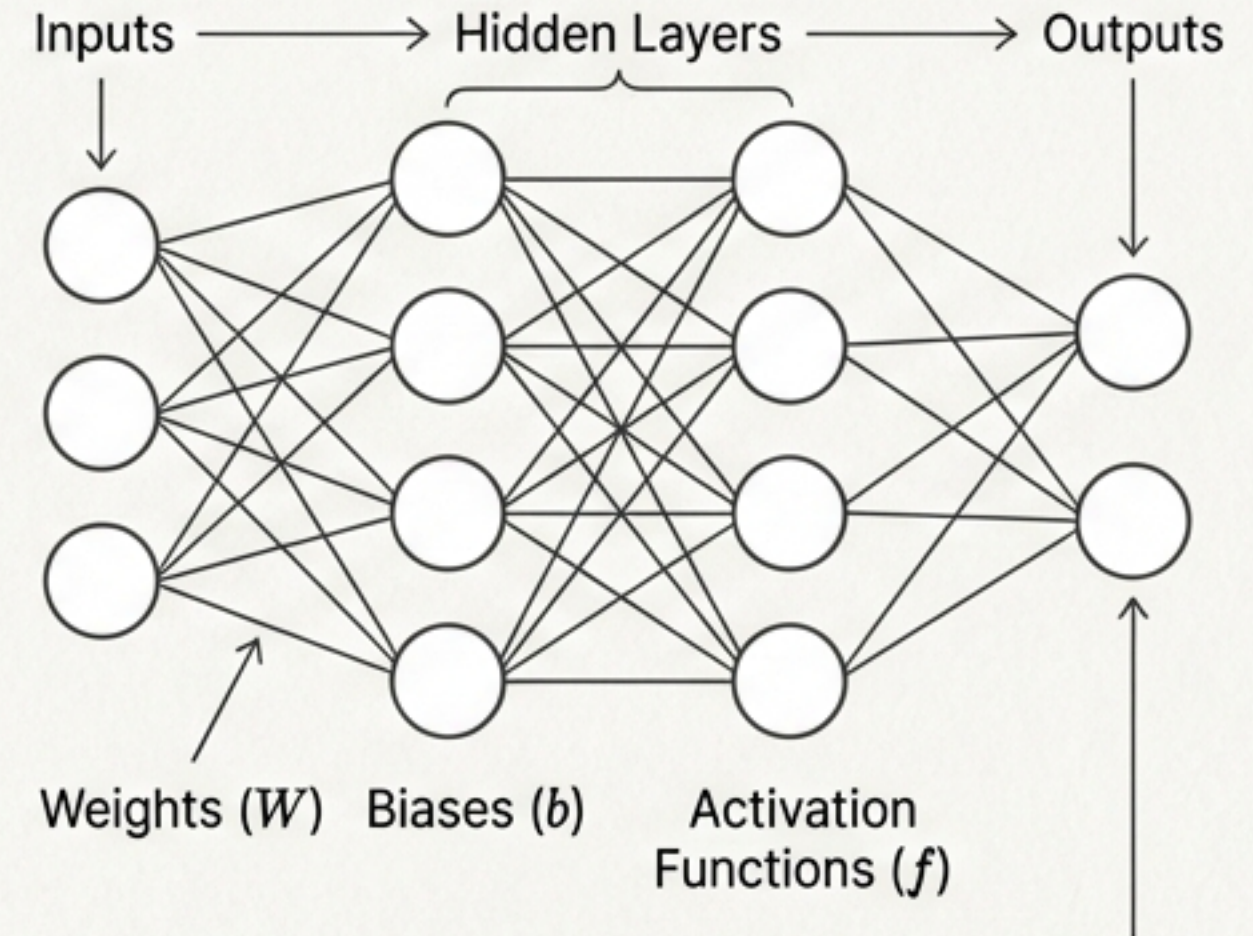
Core Concept: A single line separates data points into two classes.



The Basic Unit: The first formal mathematical model inspired by biological neurons.



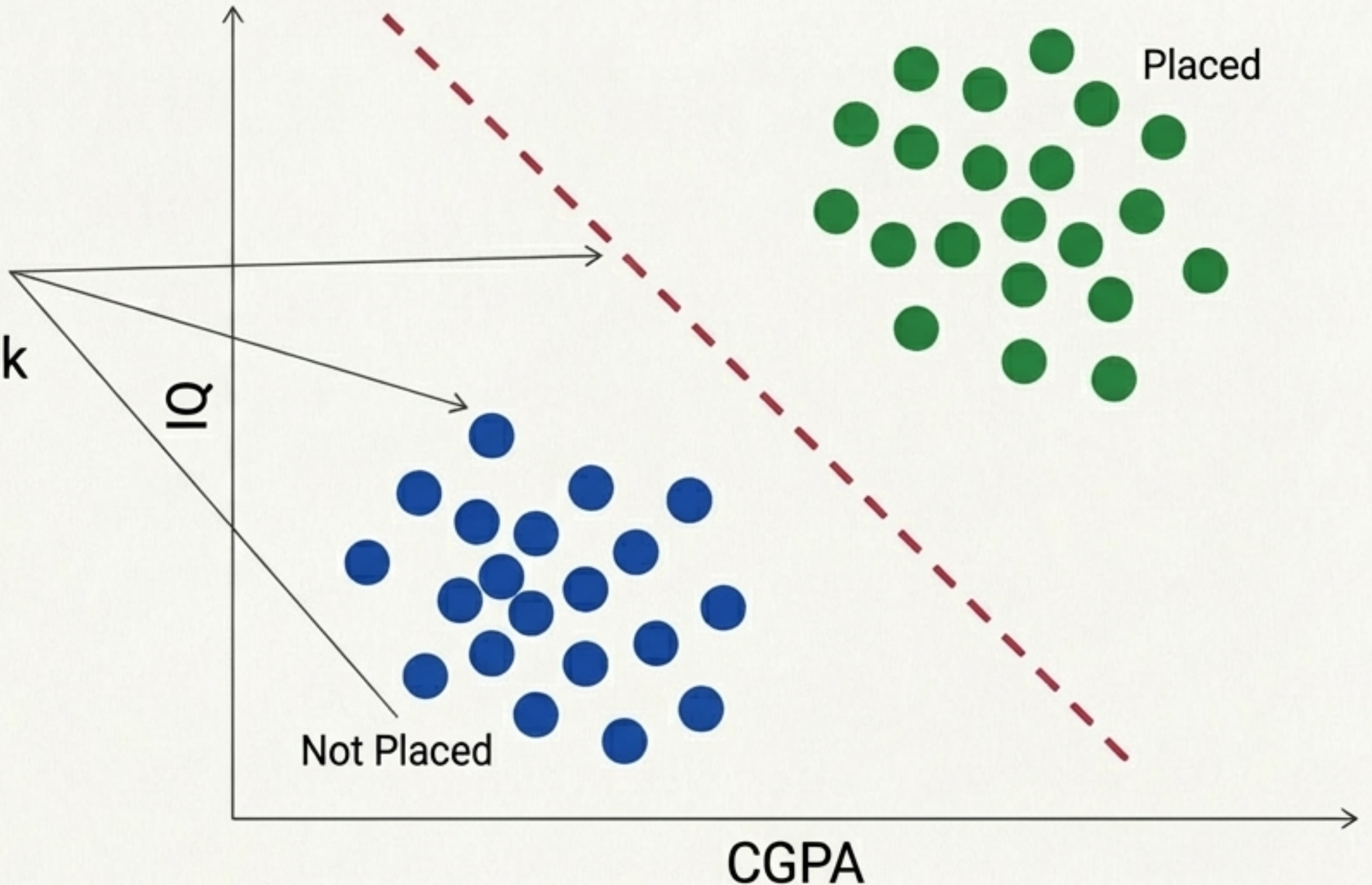
Scaling Up: Stacking multiple Perceptrons (layers) allows for learning complex non-linear relationships.



The complex network is fundamentally built upon the same linear separation logic, repeated and transformed.

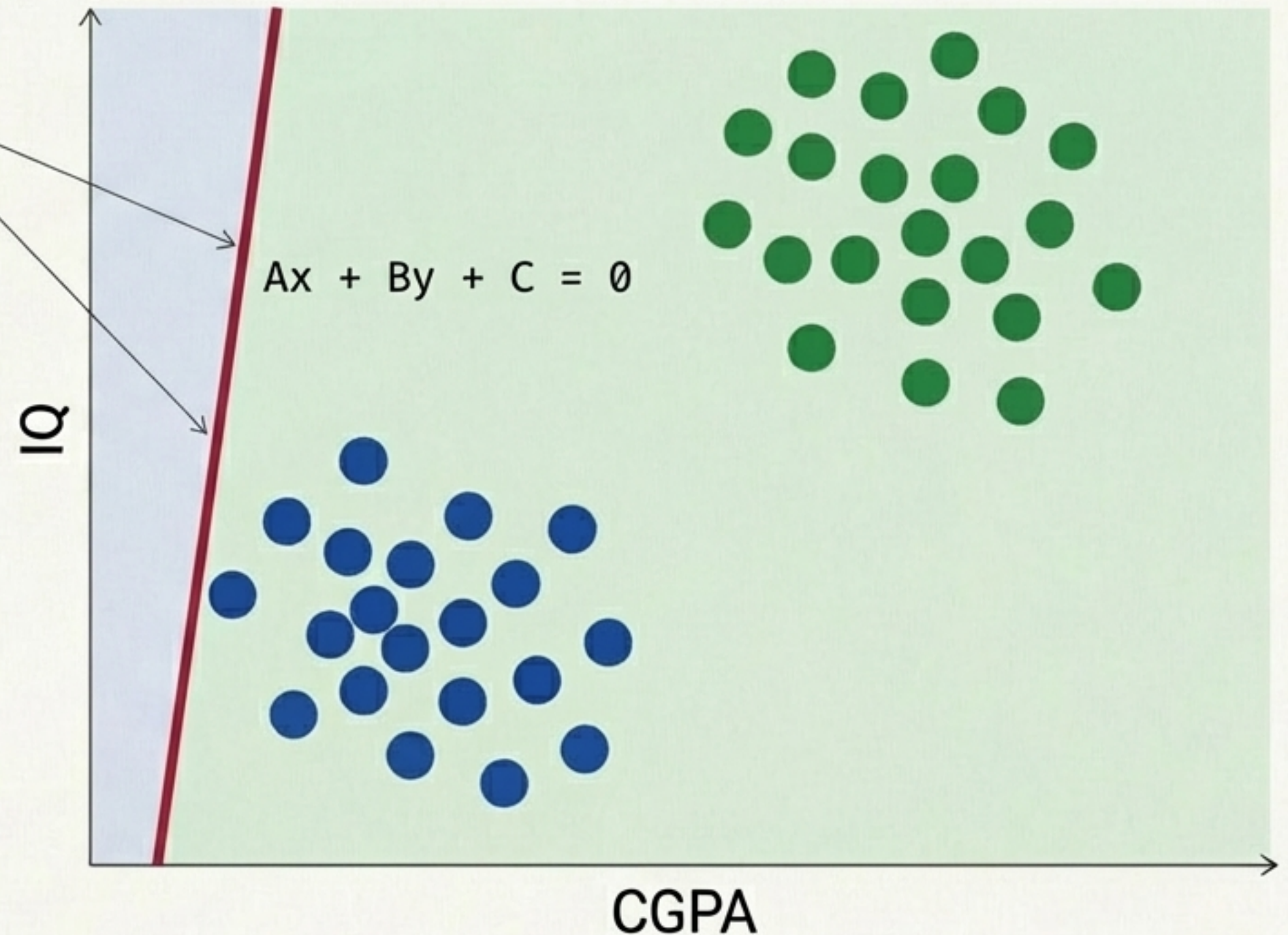
The Golden Rule: Linear Separability

- The data must be linearly separable.
- If a straight line can divide the classes (Placed vs. Not Placed), the Perceptron Trick will find it.



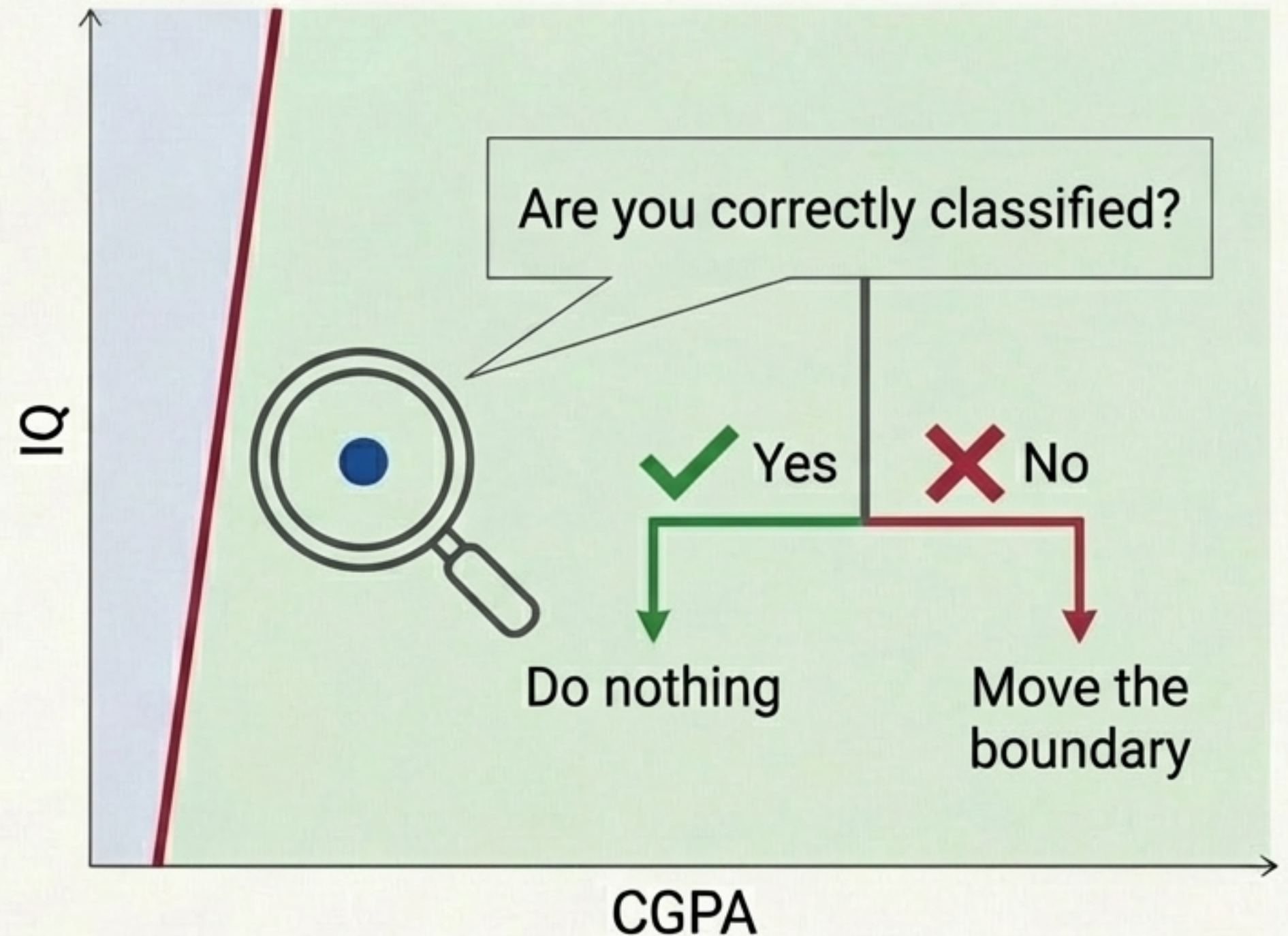
Starting in the Dark

- We begin with random coefficients ($A=1, B=1, C=1$).
- The line guesses blindly. It is completely wrong.



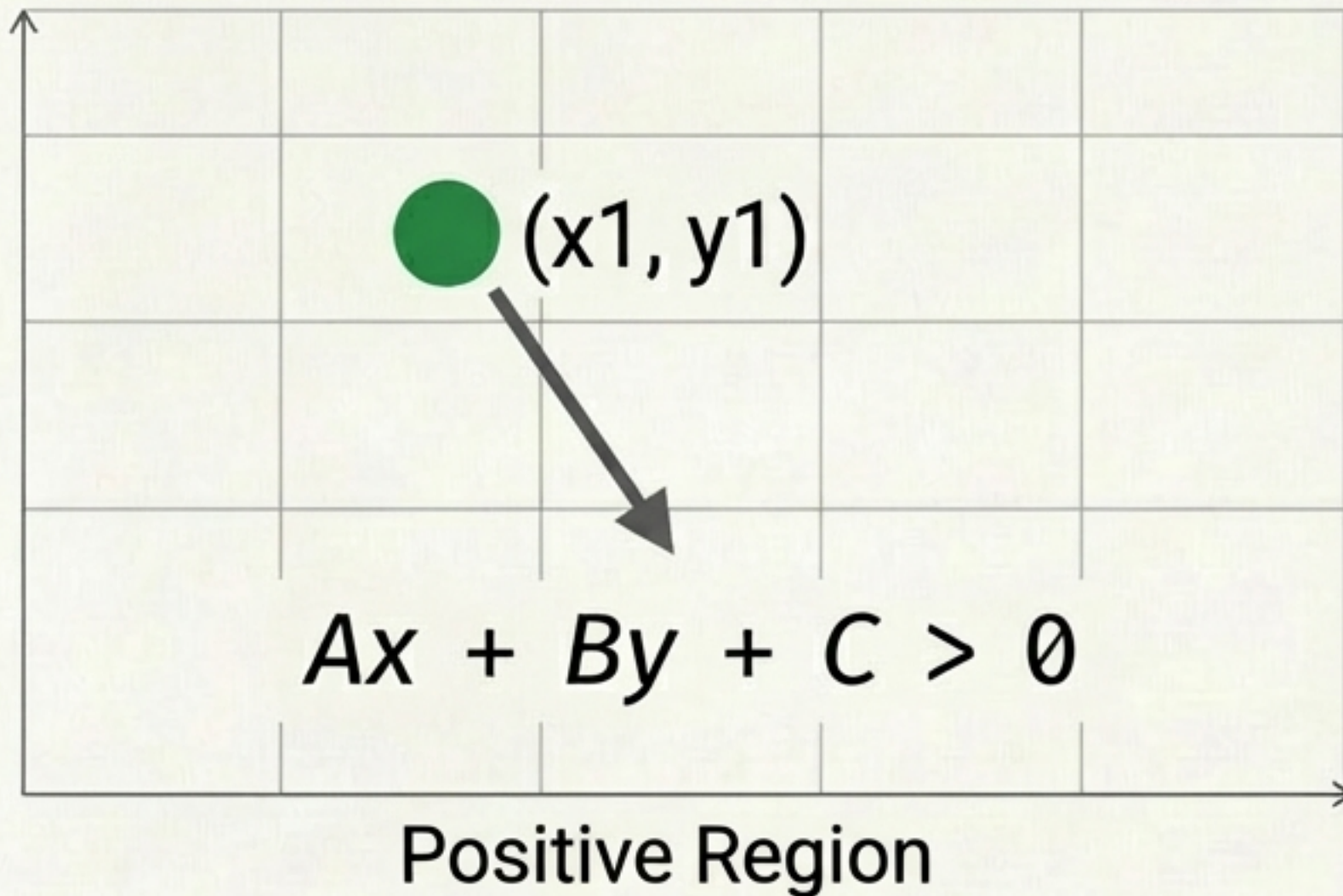
The Interrogation Loop

- Pick a random student.
- Ask the point how it feels about the line.
- Adjust the geometry only when a point complains.

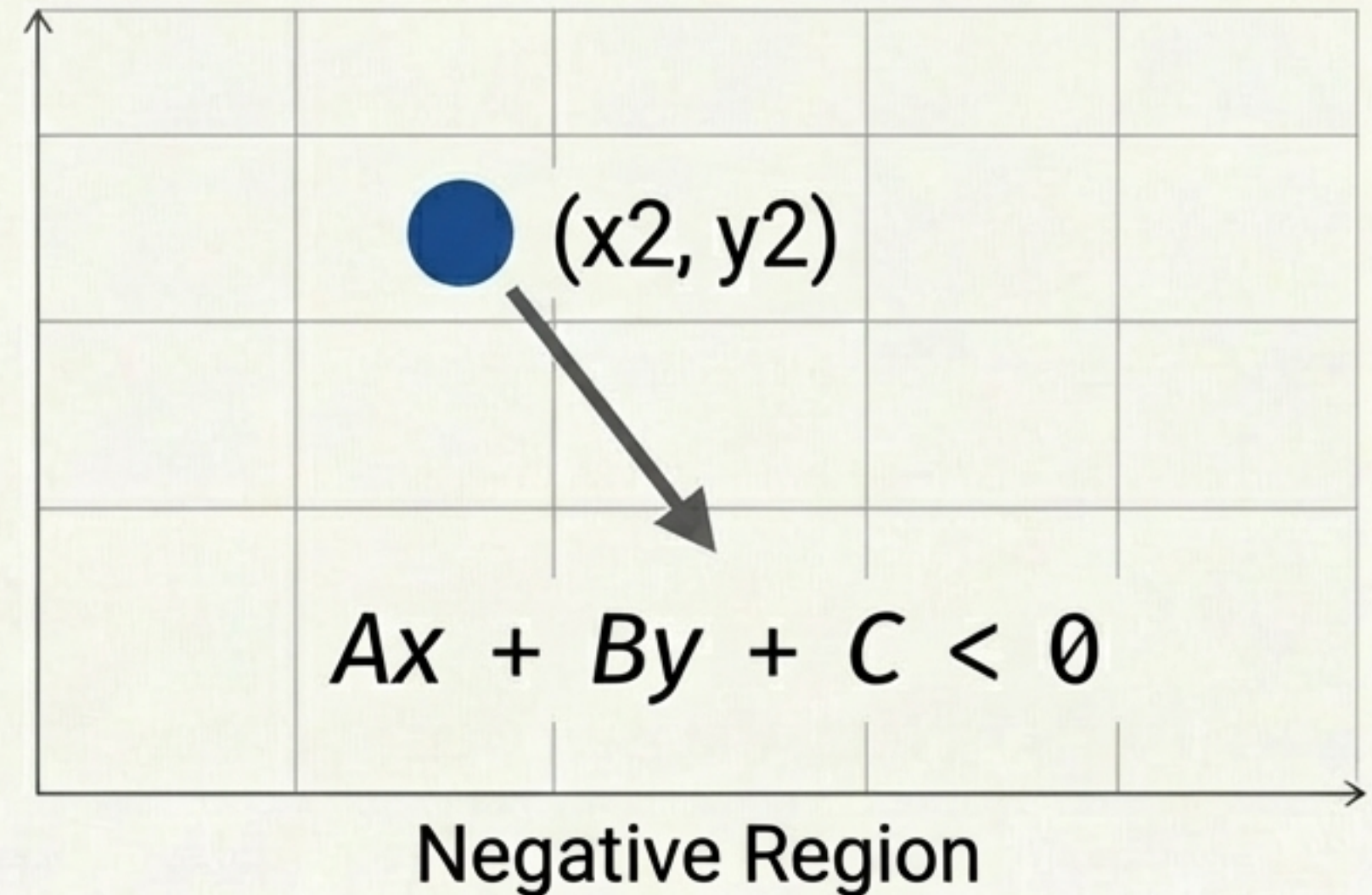


Defining Positive and Negative Regions

Positive Region: $Ax + By + C > 0$
(Predicts Placed)

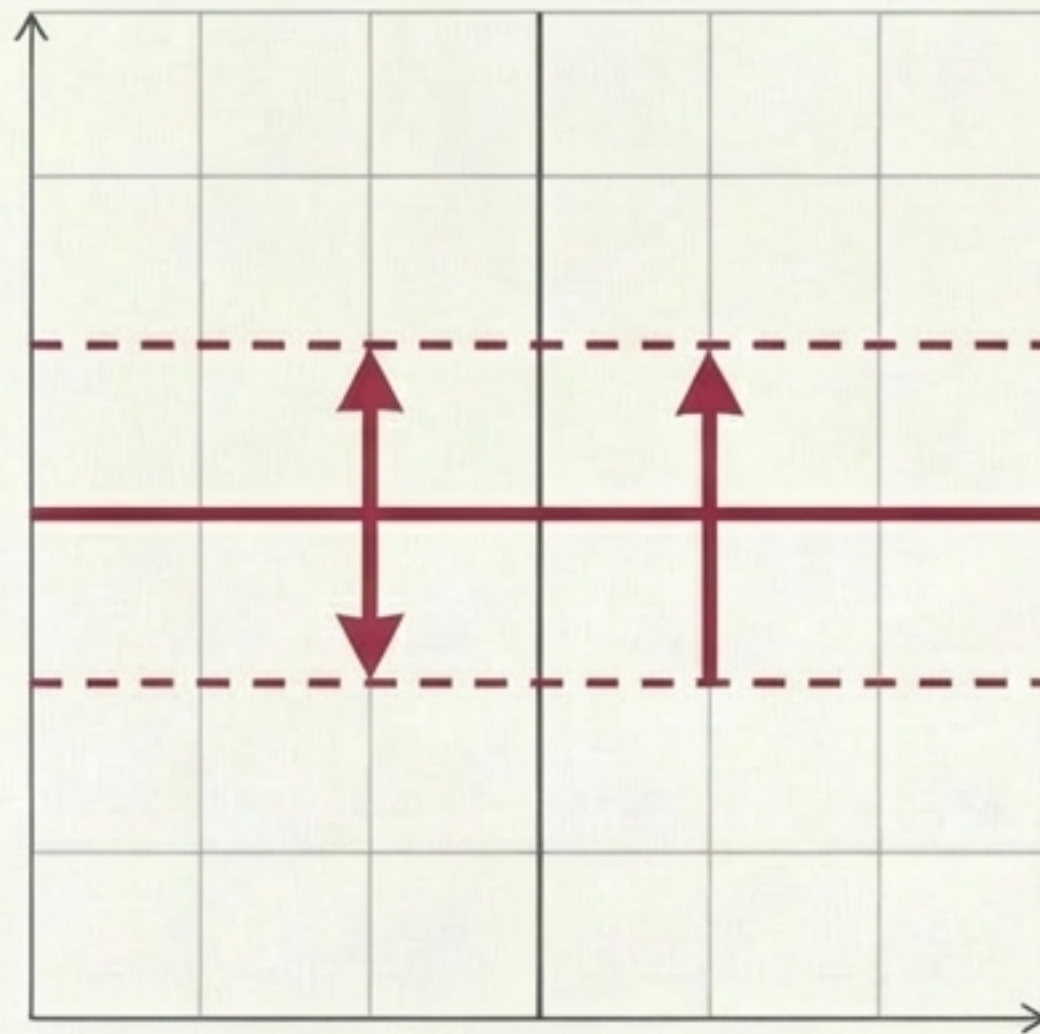


Negative Region: $Ax + By + C < 0$
(Predicts Not Placed)

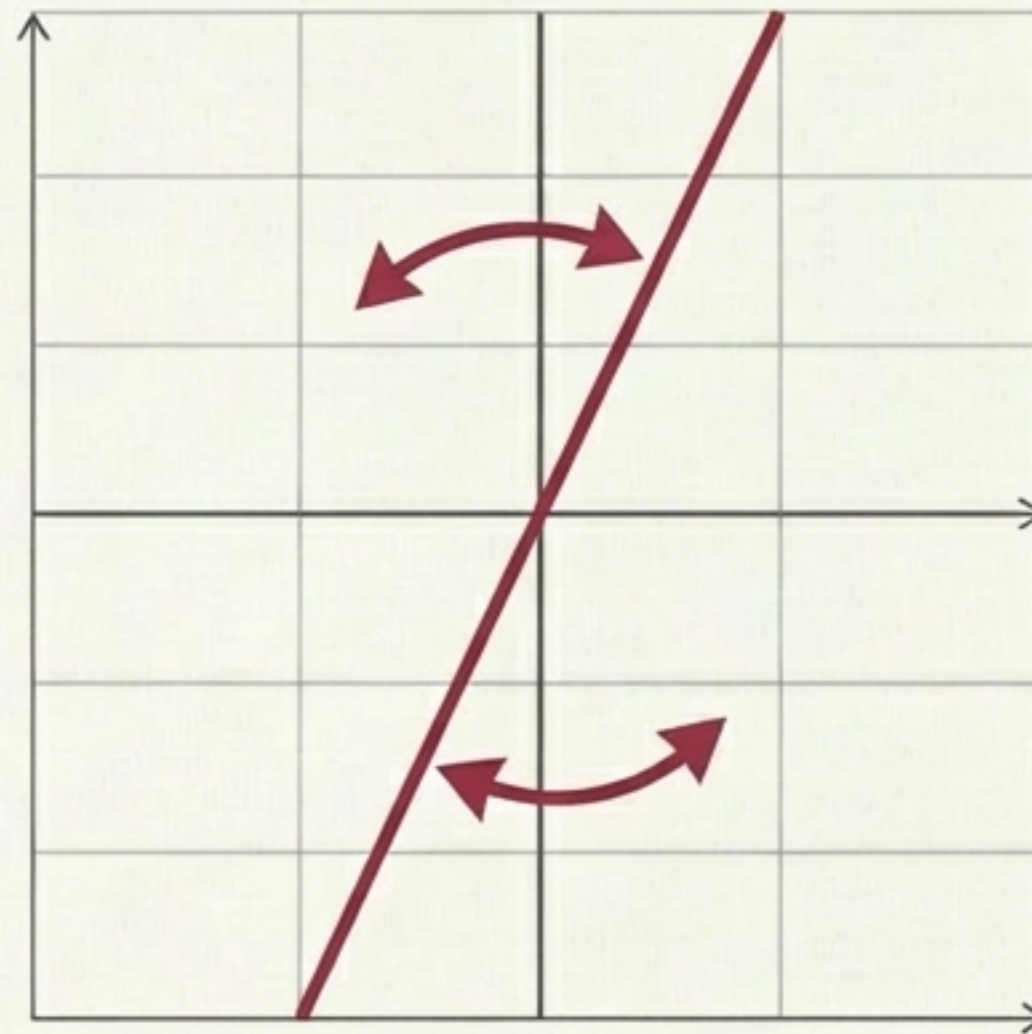


The Levers of Geometry

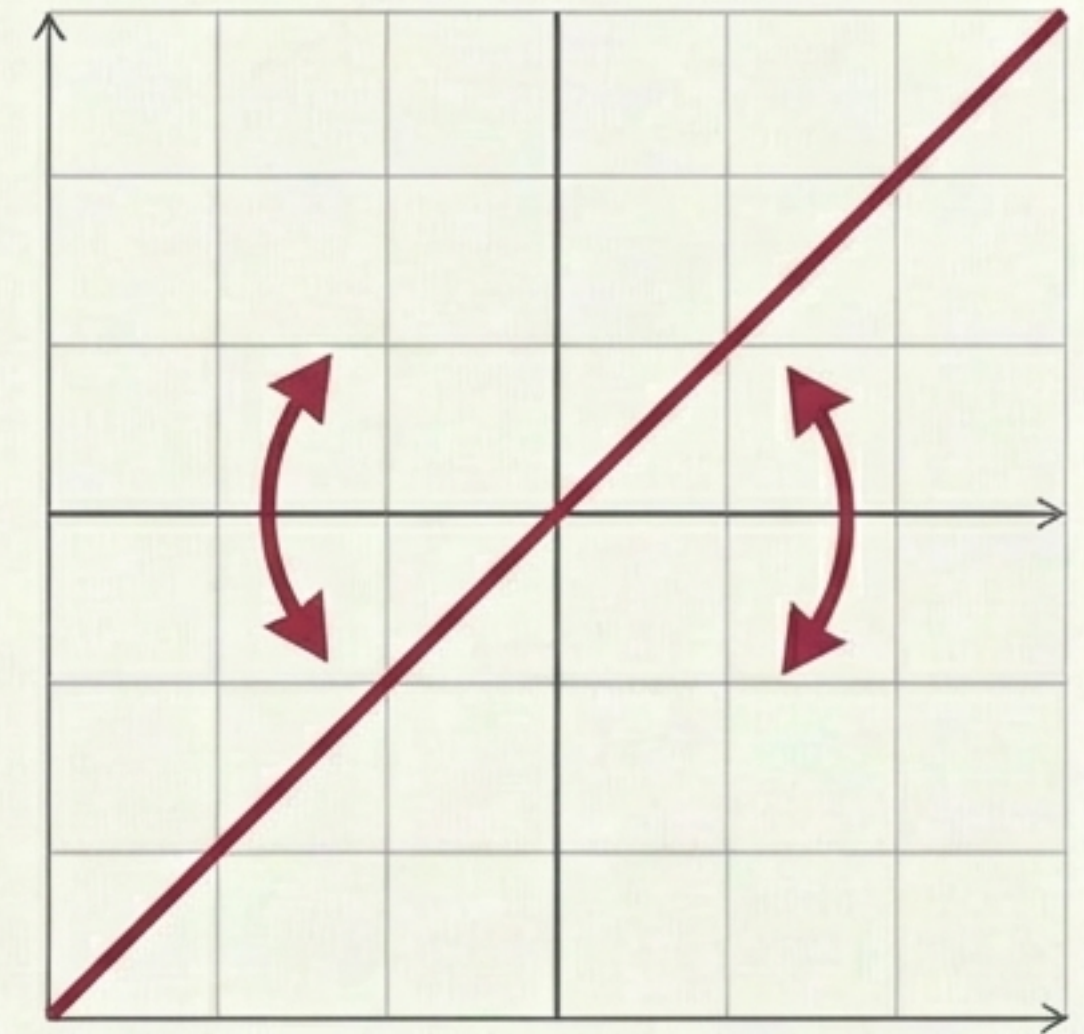
By modifying A, B, and C, we can slide and rotate the boundary anywhere on the 2D plane.



Changing C



Changing A

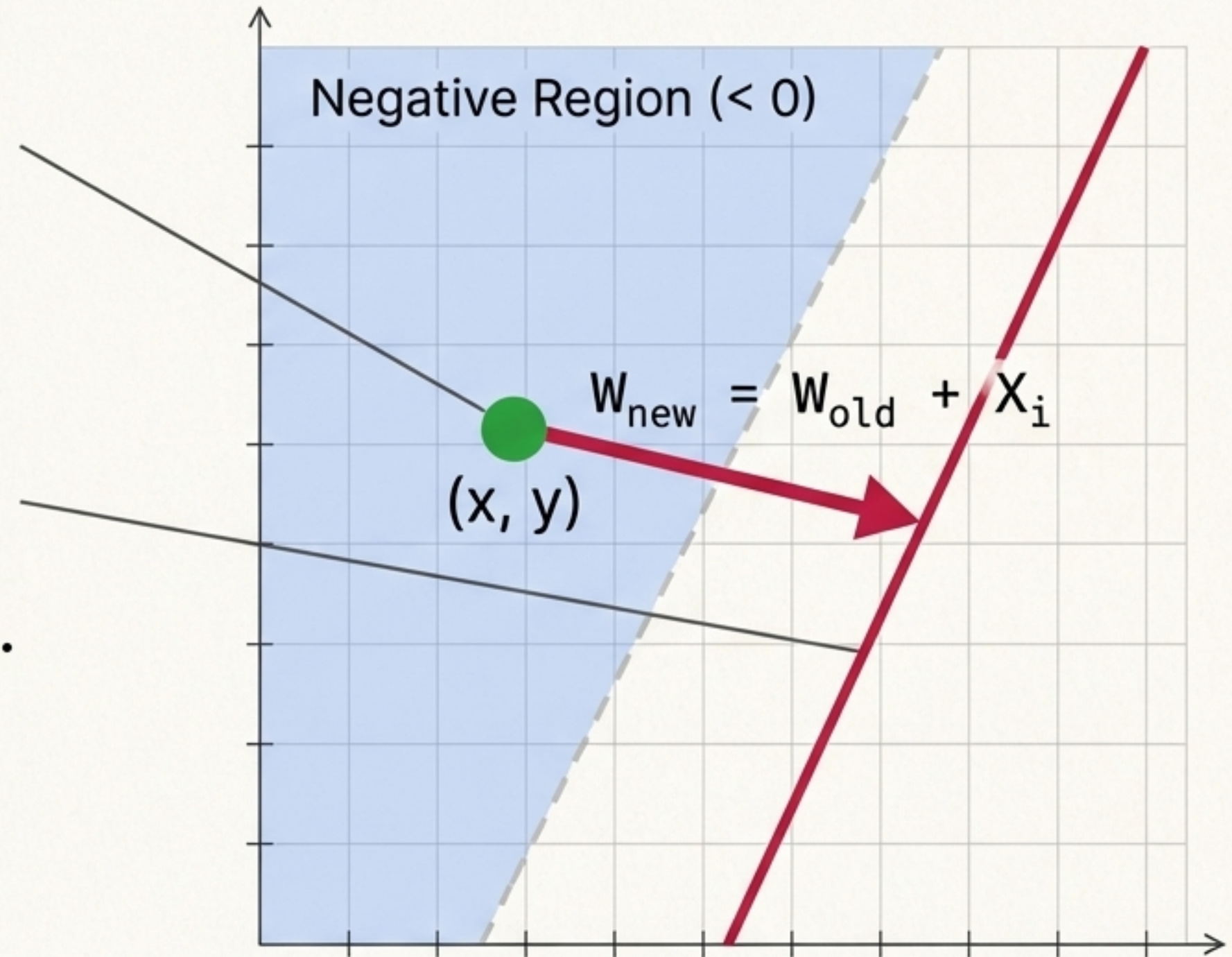


Changing B

Rescuing a False Negative

The Problem: A positive point is trapped in the negative region.

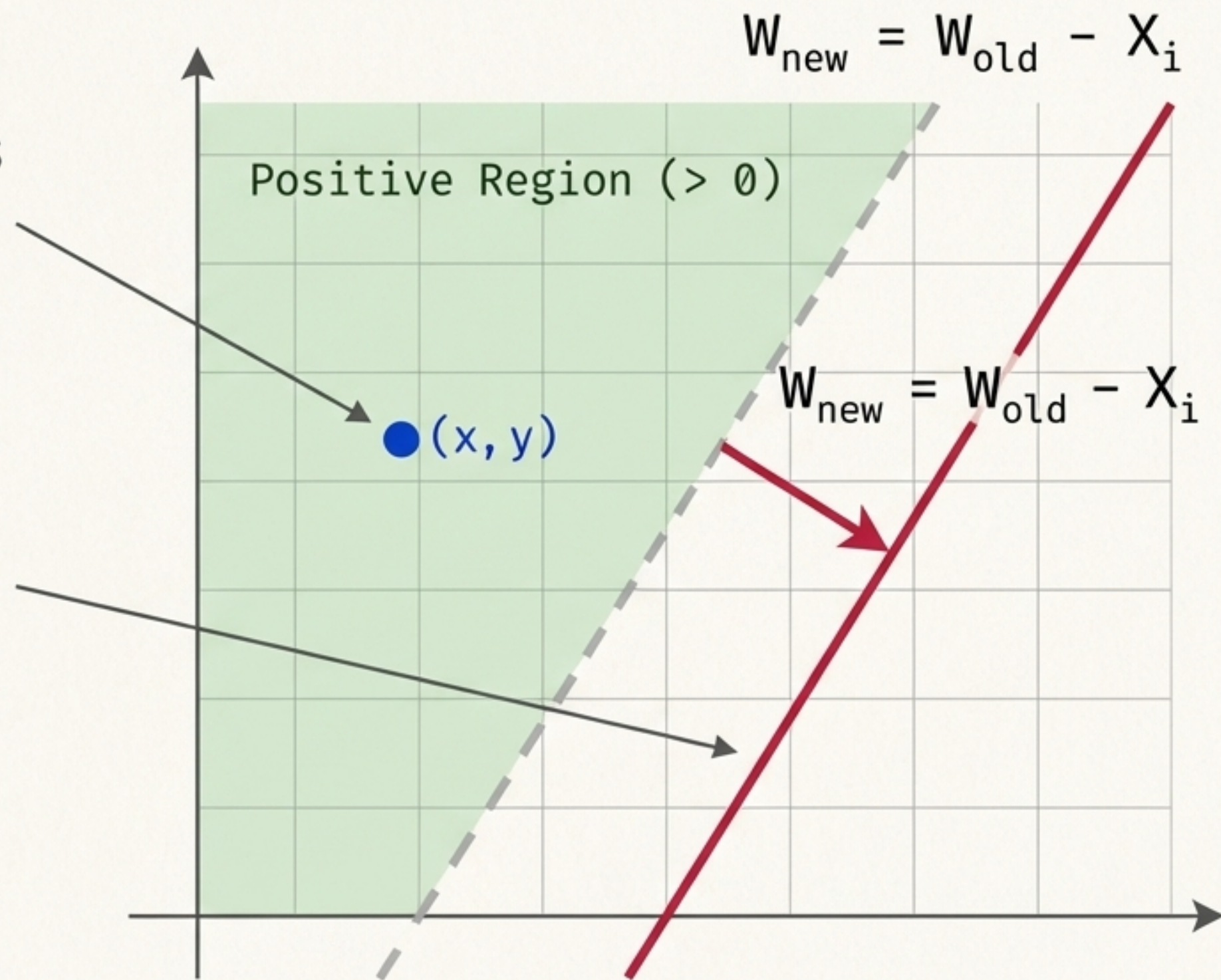
The Fix: Add the point's coordinates to the line's coefficients ($W_{\text{new}} = W_{\text{old}} + X_i$). The line is pulled closer.



Evicting a False Positive

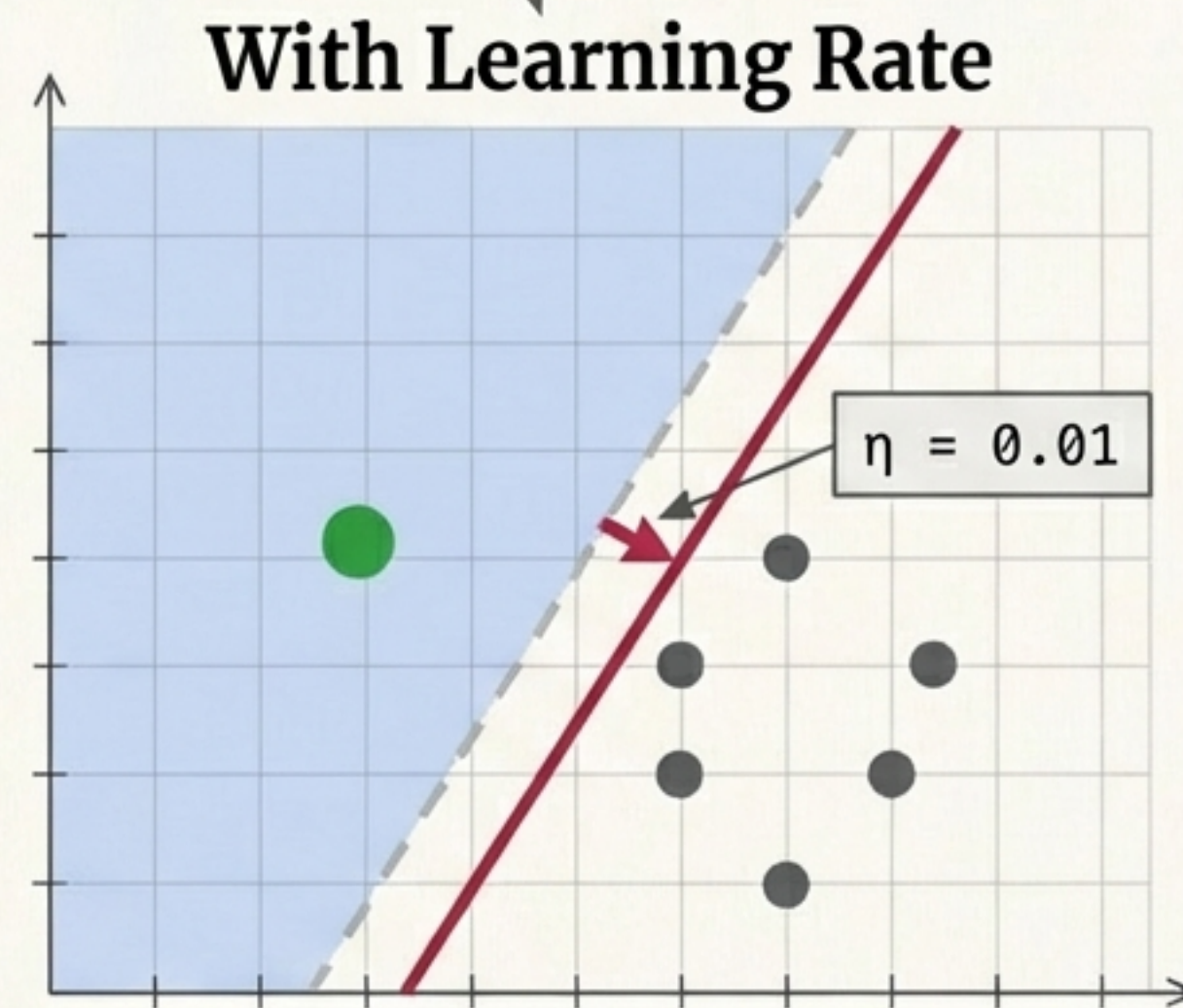
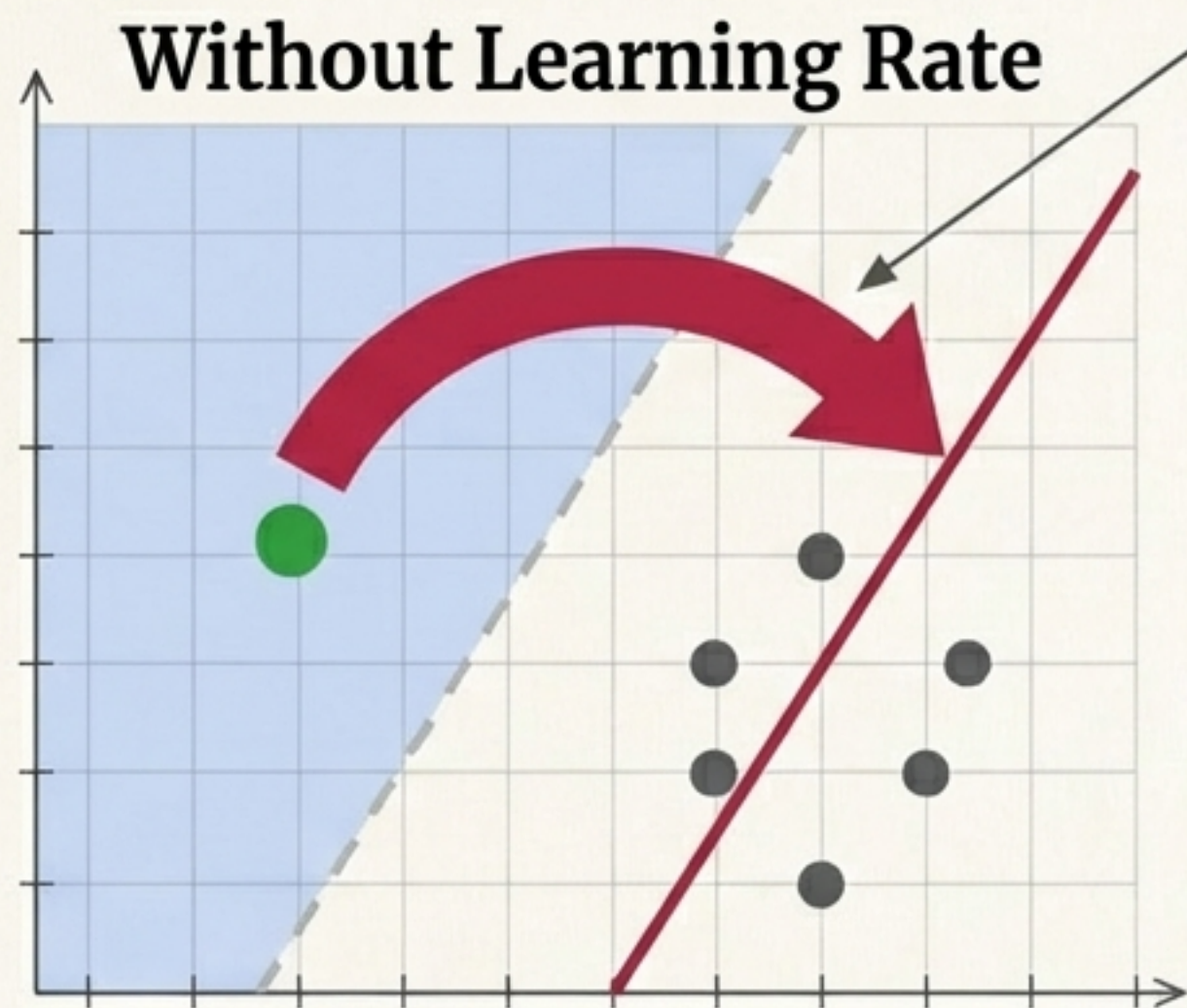
The Problem: A negative point is trespassing in the positive region.

The Fix: Subtract the point's coordinates from the line's coefficients ($W_{\text{new}} = W_{\text{old}} - X_i$). The line repels the point.



The Danger of Overshooting

- Applying massive transformations destroys previously learned progress.
- We multiply coordinates by a tiny Learning Rate ($\eta = 0.01$) to inch the boundary forward safely.



The Clunky Algorithm

This human-readable logic works perfectly.

But it relies on multiple conditional checks for every single misclassification type.

```
if point == positive and prediction == negative:  
    add coordinates  
if point == negative and prediction == positive:  
    subtract coordinates  
else:  
    do nothing
```


The Matrix of Four Cases

By subtracting the predicted value from the actual value ($y_i - y_{\text{hat}_i}$), we inherently generate the exact action required.

		Predicted Value (y_{hat})	
		1	0
Actual Value (y)	1	$(1 - 1) = 0$	$(0 - 1) = -1$ (Negative scalar)
	0	$(1 - 0) = 1$ (Positive scalar)	$(0 - 0) = 0$

The Unified Equation

No “if” statements required.

This single equation gracefully handles correctly classified points, false positives, and false negatives simultaneously.

$$W_{\text{new}} = W_{\text{old}} + \eta (y_i - \hat{y}_i) x_i$$

Learning Rate

The Directional
Scalar (0, 1, or -1)

The Student's
Data (CGPA, IQ)

The Final Optimised Loop

Human logic translates to complex code.

Mathematical geometry translates to perfect, highly-optimised simplicity.

```
For 1000 epochs:
```

```
    Pick random student i
```

```
     $W = W + \text{learning\_rate} * (\text{actual} - \text{predicted}) * \text{student\_data}$ 
```


Convergence

The loop terminates under two conditions:

1. **Epoch Limit:** We reach a predefined number of steps (e.g., 1000).
2. **Absolute Convergence:** Zero misclassifications remain on the grid.

The Perceptron Trick is complete. The foundation for Deep Learning is laid.

